

Session 1: Your first program, and the machine underneath

Read a short program, say what it prints before running it — and know what the machine is doing underneath.

PART 1

Admin

Everything you need to survive the semester
— delivered once, now, so it never interrupts
anything again.

The practical part first

Fifteen minutes of admin, then we start properly.

All of it is on the course site as well, so you do not need to write anything down.

unil-dsap.github.io

We vote a few times today. Open **wooclap.com/RSOMLOO** on your phone or laptop now, and leave it open.

Who is in front of you

Pavel levlev — lectures, and writes the exams.

Anna Smirnova — workshops and course materials.

Jinran Wang — workshops and support.

Clémence Choukroun — workshops and support.

All of us: `firstname.lastname@unil.ch`

Course questions go to the teaching assistants first. Registration and grade-record questions go to the Master's office — we cannot fix those.

When we meet

Mondays. Thirteen sessions, 14 September to 14 December.

Mondays 12:30–16:00 · Internef 263

One Monday is missing. **21 September is the Jeûne fédéral**, a public holiday, and it lands immediately after today. We come back to what that costs you at the end of this block.

The shape of the semester

Sessions	Dates	What
1–6	Sep 14 – Oct 26	Python — the language, and your files
7	Nov 2	Midterm , in class
8–13	Nov 9 – Dec 14	Data science — real data, and what you may honestly say about it

The final exam is in the January exam session.

Every Monday is two sessions

The lecture — this one. It teaches the language. You read code, predict what it does, and find out whether you were right.

The workshop — it teaches the work. You open a real workspace, run real code, break it deliberately, ask an assistant for help, and judge what comes back.

The exam is written from both.

Where everything lives

The course site — unil-dsap.github.io. Slides, workshop sheets, every handout. Each session appears there after the Monday it is taught.

Moodle. Announcements, both exams, and anything that affects your grade. If it is not on Moodle, it does not count.

Nuvolos. The computer you will write on. Next slide.

Nothing important exists only in this room.

The computer you will use

Nuvolos — VS Code in your browser. Nothing to install, nothing to configure, free for you.

Open the enrolment link — it is on Moodle and on the course site — sign in with the email you use for Moodle, and click **VSCode**. Python and every library we will touch this semester are already inside.

One identical machine for all of us. That is the entire point: when something breaks, it breaks the same way for everybody, and we fix it in one go instead of debugging two hundred laptops.

Enrol before the workshop. Two minutes now, twenty in a room of 250. Your own laptop works too; the workshop sheet assumes Nuvolos.

What you need to know already

Nothing.

No programming experience is assumed anywhere in this course — not one line, not one command.

You need a UNIL login and the willingness to be wrong in public for a few minutes each week.

Artificial intelligence: use it

This course does not ban chatbots. It is largely **about** working with them without being fooled by them.

You will use an assistant in nearly every workshop. Any free one — your choice. The course requires no particular tool, provides none, and every graded path works on a free account.

Two rules that hold all semester:

- **Never paste personal data** — names, grades, anything private. Our data is synthetic on purpose.
- **You are responsible for what you keep.** The model wrote it; you handed it in; it is yours now.

Except in the exam

Both exams are in person, proctored, on your own laptop.

Any other device, application, online service or assistant is fraud under Faculty regulations.

How the grade is built

	Weight	When
Midterm	1/3	Session 7 · 2 November · in class
Final exam	2/3	January exam session · date set by the Faculty

Details of the format come later in the semester, in good time.

The next two weeks

21 September is the holiday, so our next session is **28 September** — a fortnight from now. For a first programming course that is a long silence, at the most fragile moment of the semester.

So do not leave today's material until the night before. Open the workspace again this week, even for ten minutes, and run something. Hands stay warm or they go cold.

If you get stuck

In the workshop. That is what it is for, and a question that takes thirty seconds in person takes thirty minutes by email.

Then the teaching assistants, at `firstname.lastname@unil.ch`.

Registration problems and grade records: the Master's office. We genuinely cannot help with those.

That is the admin

Now the part that actually matters.

PART 2

Writing your first program

What a program is, what a variable is, and the one skill that everything else in this course rests on: reading code and knowing what it does.

Let's be honest about who is in this room

Some of you have never written a line of code.

Some of you have written thousands, and are already wondering whether this hour is worth your time.

Both of you are correct to worry. So let me tell you the plan.

One word first: Python

Thirty seconds, for anyone who has never met it. Everyone else, indulge me — the rest of today assumes nothing.

Python is a programming language — a small fixed vocabulary, and rules for combining it, that a computer will carry out.

A program is plain text in a file. Nothing more. The computer reads that file from the top and does what each line says, in order.

That is the whole idea. It is the language this course is taught in.

Why this language — and why it stays

It reads closer to English than the alternatives, and it is free. True, and not the reason.

The reason is libraries. A library is code somebody else already wrote and published so that you never have to: one line to pull it in, and their work is yours. `pandas` for tables, `scikit-learn` for models, `PyTorch` for neural networks.

The public shelf holds **892,078** of them, and grows every day.

Which is why it is not going anywhere

In data science and machine learning, Python is not one option among several. It is where essentially all of the work happens.

A new method arrives as a Python library first, because that is where the practitioners already are — which brings the next practitioner, and the next method. The shelf grows because it is already the biggest.

A loop like that does not reverse because a nicer language turns up. What you learn here will still be the default when you graduate.

The plan

If you have never programmed: everything today is new, and it is supposed to be. We move slowly. Nothing is assumed.

If you have programmed: today contains three things that regularly catch people with five years of Python. They are hidden in plain sight, in code you would swear you understand. I will not label them in advance.

Find them. If you find all three before I show them, come tell me after class and I'll know who I'm dealing with.

The one thing I actually want

Not that the beginners survive and the experienced are bored.

That every person here leaves with something they did not walk in with.

That is a real constraint on how I teach, and you should hold me to it.

If a session gives you nothing, that is a bug — tell me.

Now: what is this course for?

The obvious answer is **"learning to write Python."**

That answer is about five years out of date. Let me show you what changed, because it changes what we do all semester.

What "AI-assisted coding" means

No assumptions — some of you have never done this.

You open a chat window. You type, in English:

"read a csv of stock positions and print the total value."

It replies with fifteen lines of Python. You paste them into a file, run them, and something happens.

That is it. That is the whole thing. Millions of people now write software this way, including professionals.

So why are you here?

Fair question. If a machine writes the code, why learn to write it?

Here is the thing about those fifteen lines:

They will run. They will almost always run.

Whether they compute **what you asked for** is a different question, and nothing on your computer checks that.

There is no meaning-checker

Python checks **spelling**. If you write `pirnt`, it stops.

Python checks **impossible actions**. Divide by zero, it stops.

Python does **not** check whether you multiplied by the right number, used last month's price, or dropped three rows. Nothing does.

No error appears. You get a number, and the number is wrong.

The only thing standing between you and that number is a person who can read the code.

Which is now the job

You will read far more code than you write. Code from a model, from a colleague, from yourself six months ago.

And reading is **harder** than writing. Writing, you only need one idea that works. Reading, you must reconstruct someone else's idea from its residue — and decide whether to trust it.

So how do you get good at reading?

Here is the part people get wrong.

You do not get fluent at reading by reading. You get fluent at reading by **writing** — a lot, by hand, badly at first.

Writing forces you to make every small decision the author of that code made. Once you have made those decisions yourself, you recognise them instantly in someone else's work. Reading speed follows writing practice alarmingly fast.

Therefore

**You will write a lot of code by hand in this course,
in order to become someone who can read it.**

That is the bargain. When you are typing out something a model could have produced in one second, that is not busywork and it is not nostalgia — it is the only known way to build the judgement that makes you worth more than the model.

Now let's write some.

Something concrete to point at

We need actual numbers, or none of this means anything.

A tiny portfolio will do:

```
ticker, shares, price  
AAPL, 10, 190.50  
NESN, 4, 102.30  
V00, 2, 512.00
```

Today we value **one** of those positions. That is the whole ambition for the next half hour, and it is enough to break your intuition twice.

A program is a list of instructions

```
1 ticker = "AAPL"  
2 shares = 10  
3 price = 190.50
```

Three instructions. Python does line 1, then line 2, then line 3.
Top to bottom, one at a time, no exceptions, no cleverness.

That is genuinely most of it.

Read this cold

Do not worry about being right. Just predict.

```
1 ticker = "AAPL"
2 shares = 10
3 price = 190.50
4
5 value = shares * price
6 print("value at", price, "is", value)
7
8 price = 200.00
9 print("price is now", price)
10 print("value is still", value)
```

What does line 10 print? Vote now — no discussion, no running it.

1905.0 · 2000.0 · 200.0 · it crashes

wooclap.com/RSOMLOO — we come back to this in ten minutes.

What a variable is

```
1 shares = 10
```

`shares` is a **name**. `10` is a **value**. The `=` sticks the name onto the value.

Say it out loud as "**shares gets 10**", never "**shares equals 10**". That one word is responsible for more confusion than any other in this course.

= is not equality

In mathematics, $x = x + 1$ is a false statement with no solutions.

In Python it is an instruction, and a perfectly ordinary one:

```
1 count = 5
2 count = count + 1
```

Read line 2 right to left: **take what** `count` **currently is, add one,**
put the result back under the name `count` . Afterwards `count` is 6.

Time passes in a program. That is the whole idea.

Excel lied to you

You know spreadsheets. In Excel, if `C1` holds `=A1*B1` and you change `A1`, then `C1` **updates by itself**. The cell remembers the **recipe**.

Python does not do this. A variable remembers the **result**, not the recipe.

```
1 value = shares * price
```

This does not mean **"value is shares times price, forever."**

It means **"work out shares times price right now, and call the answer value."** Then it is done. It never looks back.

Tracing by hand

The single most useful thing you will learn today. One row per line, one column per name. Walk it like the machine.

line	ticker	shares	price	value
1	"AAPL"	—	—	—
2	"AAPL"	10	—	—
3	"AAPL"	10	190.50	—
5	"AAPL"	10	190.50	1905.0
8	"AAPL"	10	200.00	1905.0

```
1 ticker = "AAPL"
2 shares = 10
3 price = 190.50
4
5 value = shares * price
6 print("value at", price, "is", value)
7
8 price = 200.00
9 print("price is now", price)
10 print("value is still", value)
```

Line 8 changes `price`. Look at the `value` column. Nothing moved.

Read it again

```
1 ticker = "AAPL"
2 shares = 10
3 price = 190.50
4
5 value = shares * price
6 print("value at", price, "is", value)
7
8 price = 200.00
9 print("price is now", price)
10 print("value is still", value)
```

Second time through, it should feel obvious. **That feeling is the lesson.** No amount of raw intelligence gets you there cold, because `value = shares * price` genuinely **could** mean "value is always shares times price" — that is exactly what it means in Excel. You had to be told which of the two it is. Now you know, and you cannot unknow it.

What it actually prints

```
$ python ledger.py  
value at 190.5 is 1905.0  
price is now 200.0  
value is still 1905.0
```

Line 10 prints **1905.0**. The price is 200, the value is computed from 190.50, and Python is entirely happy about it.

Look at how many of us voted `2000.0`.

One of these is wrong. Both of them run.

Version A `python wrong.py` → `position value: 1905.0`

```
3 value = shares * price
4 price = 200.00
```

Version B `python right.py` → `position value: 2000.0`

```
3 price = 200.00
4 value = shares * price
```

Two lines swapped. No error either way.

Only one of them is your portfolio.

The three ways code fails

1. **It won't start** — you typed something Python can't parse. Loud, instant, harmless. You will do this constantly and it costs nothing.
2. **It crashes** — it starts, then hits something impossible and stops with a message. Also loud, also honest, also fixable.
3. **It runs and lies** — no error, wrong number.

The slide before this was type 3. **Type 3 is what this course is actually about**, and it is the only kind a model produces fluently.

The first thing hiding in plain sight

```
1 print(0.1 + 0.2)
```

```
0.30000000000000004
```

This is not a bug. `0.1` cannot be written exactly in binary, any more than `1/3` can be written exactly in decimal. Your computer stores a very close neighbour and does arithmetic on that.

Consequence: `0.1 + 0.2 == 0.3` is **False**. Never compare money with `==`.

The second and third

```
1 print(round(0.5), round(1.5), round(2.5), round(3.5))
```

```
0 2 2 4
```

`round` breaks ties toward the **even** number — the banking standard, because always-rounding-up biases a portfolio upward. Not what most people expect.

And addition is not associative:

```
0.1 + 0.2 + 0.3 -> 0.6000000000000001  
0.3 + 0.2 + 0.1 -> 0.6
```

Same numbers. Different order. Different answer.

Why I showed you those

If you came in experienced, those three are the point: you have probably shipped code that assumed otherwise.

If you came in new — you have just seen that "the computer got it wrong" is a thing that happens to everyone, forever. It is not a beginner problem. Welcome.

Nobody in this room is done learning this.

PART 3

What is underneath

You have written a program. Now: what is a computer, what is programming, and where does the thing you just wrote actually sit?

Stop. What did we actually just do?

You read a program. You predicted it. You caught it lying.

That is the whole job, and you did it in twenty minutes.

But you should know what is underneath — not because you will ever touch it, but because every strange thing this course shows you comes from down there.

Programming, from a distance

Strip away the vocabulary and there are only three things.

Data	facts, written down in a form a machine can hold — a price, a name, a photograph
Operations	things that turn data into other data — add, compare, sort, search
Control	saying which operation happens, and when — in order, or only sometimes, or repeatedly

A **program** is those three, arranged on purpose. Your ledger held data (`shares`, `price`), did an operation (`*`), in an order that mattered.

Everything else — every library, every language, every job title — is built from these. What changes is only how much someone has already built for you.

The machine knows two things

A computer is built from switches. Each one is either off or on.

Everything else follows from that. But what **is** a switch? Thirty seconds, because the whole stack sits on this one object.

A switch is a tap for electricity

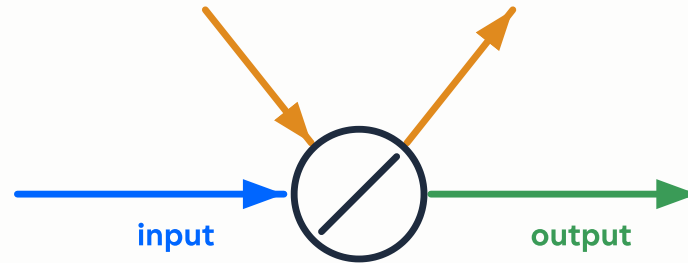
A **transistor** is a switch: current passes through it, or it does not.

a transistor is a switch



current passes through, or it does not

thrown by a third wire
control



current here holds the switch open

What makes it special is **how** it is thrown. Not by a hand — by a third wire. Current in the control wire holds the switch open; cut that current and the switch closes.

And the control wire is just a wire. So **one switch can be thrown by**

So a switch remembers one fact

A single switch, left in a position, holds one piece of information: on or off, 1 or 0. That is a **bit** — the smallest thing a computer can know.

One bit is almost nothing. But put eight in a row and there are 256 possible settings; put sixty-four in a row and there are more settings than there are grains of sand on Earth.

So the first question is not "how does it think". It is: **how do you write down a price, or a word, as a row of switches?**

Storing a number

You already know the rule. In decimal, 190 is $1 \times 100 + 9 \times 10 + 0 \times 1$ — each position worth ten times the one to its right.

Binary is the same rule, each position worth **twice** the one to its right:

128	64	32	16	8	4	2	1	
1	0	1	1	1	1	1	0	= 190

$128 + 32 + 16 + 8 + 4 + 2$. Not a code, not a trick: arithmetic in a different base. Eight switches now hold a number.

Storing text — and everything else

Eight bits give 256 patterns, and there are fewer than 256 letters, digits and punctuation marks on a keyboard. So people simply agreed on a table: **A** is 65, **B** is 66, space is 32.

The same move works for anything you can list. A colour is three numbers. A sound is a number many times a second. An image is a grid of colours.

Storing X means: agree on how to write X as numbers. Once agreed, it is bits, and the machine can hold it.

Your ledger, as the machine sees it

`ledger.csv` is **62 bytes** on disk. Here are the first six:

116	105	99	107	101	114
t	i	c	k	e	r

That is the word `ticker`, which you typed. There is no text down there — only numbers, and an agreement about what they mean.

Take the byte `01000001`. It is 65, **and** the letter `A`, **and** a dark grey. **Nothing in the bits says which.** The program decides.

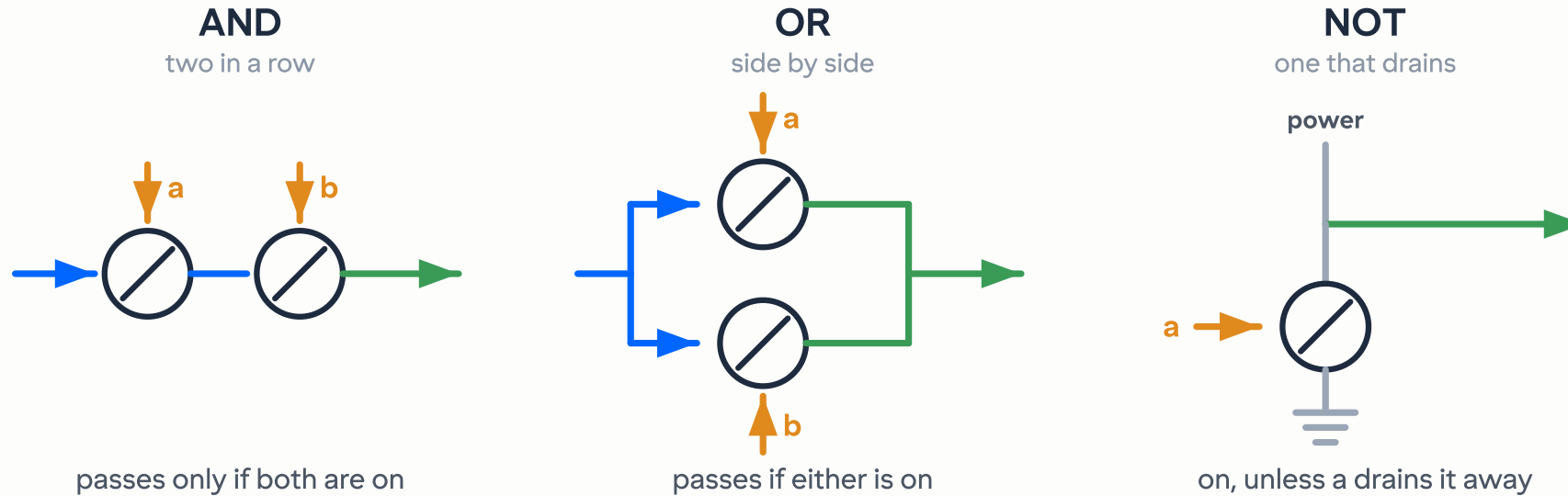
Storing is not enough — we want to **do** things

So far the switches just sit there holding values. A computer has to combine them: compare two prices, add a column, find a word.

Here is where the transistor earns its keep. Remember the one strange property: **a switch is operated by electricity**, so the output of one switch can be the handle of the next.

That means you can wire switches to each other — and a wiring pattern **is** an operation.

The simplest operations are wiring patterns



Nobody invented these. They fall out of how the wires go. Each little pattern is called a **gate** — a physical thing that performs a logical operation.

Arithmetic is the same idea, wired harder

Addition is not a different kind of magic. It is a bigger wiring pattern.

Add two bits and binary gives you $1 + 1 = 10$ — an answer needing two columns, a **sum** and a **carry**. Decide what the output should be for each of the four possible inputs, and there is a wiring that produces exactly that. It takes a couple of dozen transistors.

Then chain them, passing each carry leftward — the same thing you do on paper when a column runs past 9:

1 1 1 1 0	← carries, passed left
1 0 1 1	(11)
+ 1 1 0 1	(13)
1 1 0 0 0	(24)

And then everything else

Comparing two prices, multiplying them, sorting a column, matching a word, adjusting a pixel — every one of them is built the same way:
decide what output each input should produce, then wire switches that do it.

There is no later point where a different kind of magic arrives. The machine has exactly one trick, applied at enormous scale.

The scale is the surprising part

That adding circuit — enough for any number in your ledger — is about **1,800 transistors**.

The chip in a current Mac has **28 billion** of them. The adder is 0.000006% of the chip.

They are etched onto silicon a few dozen atoms wide and switch billions of times a second. At 3 GHz one tick is so short that **light itself travels only 10 cm** — so the chip must be small, or a signal could not cross it in time. Physics sets that limit, not effort.

Why there are so many: Moore's law

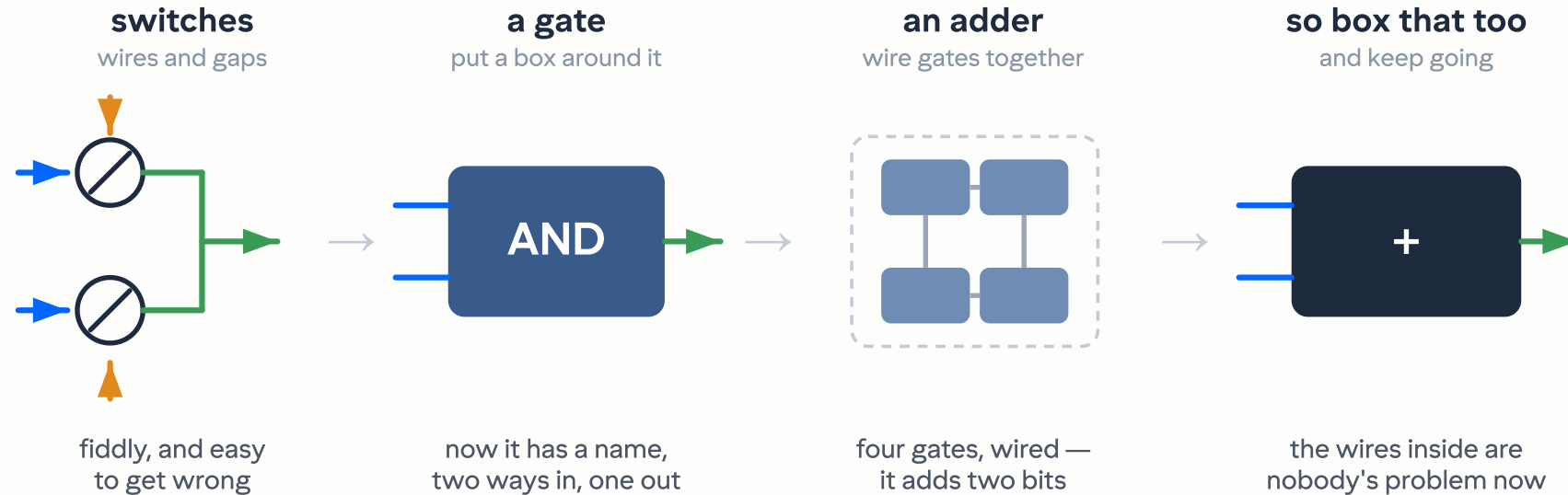
In 1965 Gordon Moore noticed the count was doubling roughly every two years, and said so in print. It was an observation about a young industry — but it became a target the whole industry then aimed at.

	transistors
1971 — Intel 4004	2,300
2024 — Apple M4	28,000,000,000

That is **12 million times** more in 53 years: a doubling every 2.25 years, held for half a century. Nothing else you own improved like this.

It is now slowing — the parts are a few dozen atoms wide, and atoms do not get smaller. Which is part of why **writing efficient code matters again**: the free speed-up is ending.

How anything big gets built



The principle: build it once, then stand on it

This is the move that makes everything above possible.

you build	and then it is just	
switches wired into a gate	"AND"	forget the wires
gates wired into an adder	"+"	forget the gates
adders and more into a processor	one instruction	forget the circuit
instructions into Python	<code>shares * price</code>	forget the instructions

Each level is built out of the one below, then **never thought about again** — it becomes a single word at the level above.

`value = shares * price` is real work all the way down. You are arranging billions of switch-flips with one sentence.

There is nothing else in there

No matter how far up you go — your ledger, a browser, a bank, a model that writes essays — go far enough down and you find the same two things, and only these two:

Transistors, and wires between them.

Not "mostly". There is no other ingredient waiting further down. What changes between a calculator and a supercomputer is how many, how small, and how cleverly arranged.

Everything you will learn this semester is a way of arranging them without having to think about them.

Prices: the same scheme, one step further

Extending binary to fractions costs nothing new. Positions to the left of the point double; positions to the right **halve**:

8	4	2	1	.	1/2	1/4	1/8
					0.5	0.25	0.125

So `190.5` is the 190 you already built, plus a single `1` in the half column. Same rule, continued rightward.

One thing is missing: where the point sits. The machine does not draw it — so it stores **where to put it**, alongside the digits.

The three parts of a price

Digits in one place, scale in another — exactly as 1.905×10^2 does in decimal:

$$190.5 = 1.01111101 \times 2^7$$

bits	holds	job
1	sign	positive or negative
11	the exponent, here ⁷	range : where the point goes
52	the digits after the leading 1	precision

Storing the scale separately lets 64 bits cover 0.004 and $4,000,000$ alike — and gives **53 significant bits**, about **15 decimal digits**.

So why is anything wrong?

Because those 53 digits have to **end**. And in base 2, some ordinary decimals never do.

Convert `0.3` to binary by hand — double it, write down the whole part, keep the rest:

```
0.3 × 2 = 0.6    → 0,   keep 0.6
0.6 × 2 = 1.2    → 1,   keep 0.2
0.2 × 2 = 0.4    → 0,   keep 0.4
0.4 × 2 = 0.8    → 0,   keep 0.8
0.8 × 2 = 1.6    → 1,   keep 0.6    ← 0.6 again
```

The remainder `0.6` has come back. Everything from here repeats, forever:

```
0.0100110011001100...
```

The same thing you already accept in decimal

You cannot write $1/3$ exactly in decimal. `0.333...` never closes. It is not a flaw in decimal — 3 simply does not divide a power of ten.

`0.3` cannot be written exactly in binary for the identical reason: 10 does not divide a power of two.

So there is no 53rd bit to stop at. The machine keeps the **closest value it can represent** and continues. Every price you type is silently replaced by its nearest available neighbour.

Which is why your ledger is half-honest

AAPL	190.50	stored exactly
NESN	102.30	stored as 102.29999999999999715783

190.50 survives: .5 is exactly one half, a power of two, so the digits terminate immediately. 102.30 does not terminate, so it is approximated.

Same file. Same column. One is exact and one is not, and nothing on your screen tells you which.

Add that NESN price a hundred times: 10229.999999999996.

That is the earlier gotcha, explained. It was never Python being careless.

So what were you actually writing?

You wrote `value = shares * price` — words, in a file. The machine cannot execute words.

Between you and the switches there is a stack, and every layer exists to let a human say something shorter:

Machine code	the bits themselves. Nobody writes this.
Assembly	one word per instruction. <code>MOV</code> , <code>ADD</code> . Brutal.
C, Rust, Fortran	human-readable, compiled — translated once, in advance, into machine code. Fast, unforgiving.
Python	human-readable, interpreted — read and executed line by line, while it runs.

Where you are standing

High up. Deliberately.

Your one line, `value = shares * price`, becomes thousands of switch operations that you will never see and never need to.

Each layer down buys speed and costs you time, clarity, and safety.
Each layer up costs speed and buys you the ability to express an idea in the time you have.

Data science lives at the top — and reaches down through libraries when it needs the speed. That is the bargain of this course.

The machine, in one slide

- **The processor (CPU)** — does arithmetic. Astonishingly fast, and astonishingly narrow: a handful of operations at a time, one after another, very quickly.
- **The graphics processor (GPU)** — thousands of much simpler arithmetic units, all doing the **same** operation on different numbers at once. Built for drawing pixels; it turns out training a model is the same shape of work — multiply an enormous grid of numbers, over and over. **That is why machine learning runs on GPUs**, and we will come back to it at the end.
- **Memory (RAM)** — the desk. Fast to reach, wiped when power goes. Your variables live here while the program runs.
- **The disk** — the filing cabinet. Slower, and it survives. Files.

The trade between two of those parts

Look at that list again. The processor **computes** things; the memory and disk **store** things. They are separate resources, and you are constantly spending one to save the other.

Say your program needs the same answer over and over. Two ways to get it:

- **work it out each time** — costs processor time, needs no space
- **work it out once and keep the answer** — costs space, saves time

Neither is right. It depends on which one you have less of.

When memory is what you do not have

The chip in a refrigerator. It reads a temperature sensor and decides whether to run the compressor. It has about **2 KB** of memory — 2,048 bytes, roughly **8 million times** less than your laptop.

The sensor does not report degrees. It reports a raw number, 0 to 1023, which has to be converted — a little arithmetic, every reading.

It could do that sum once for each of the 1,024 possible readings and keep the answers. That table would be **2,048 bytes: the entire memory**, leaving nothing for the program meant to read it.

So it recomputes, every single time. It spends processor cycles it has, to save memory it does not.

When time is what you do not have

A server answering price queries. Memory is cheap and plentiful; what is scarce is **time**, because ten thousand people are waiting.

So it does exactly the reverse: compute every answer in advance, store them all, and reply by looking them up.

Identical problem. Opposite correct decision. The only thing that changed is which resource ran out first.

This is **memory versus compute**, and it is one of the two or three questions that shape real systems. You will meet it properly in Part II, on data that does not fit on the desk at all.

What a file actually is

A disk is one enormous row of bytes. Nothing on it is naturally divided into documents — that division is bookkeeping the operating system keeps for you.

A **file** is a stretch of those bytes, plus an entry in that ledger saying **these bytes, this many, called this**. When you "save", the system writes the bytes and updates the entry. When you "delete", it usually just removes the entry.

So a file has two separate things: **contents** (the bytes) and a **name**. Your ledger's contents are those 62 numbers. Its name is `ledger.csv`.

And that bookkeeping is stored the same way

Here is the part worth pausing on. Where does the ledger of files live?

On the same disk, in the same bytes. The list of what exists is itself stored exactly like the things it lists. There is no separate magic place.

A folder is the clearest case: it is not a container holding files. It is a **stored list of names**, each paired with the location of some bytes. It has its own size, and that size changes as the list changes:

empty folder	64 bytes	
after adding ledger.csv	96 bytes	
after adding report.txt	128 bytes	
after deleting report.txt	128 bytes	← the name went, the slot stays

Which explains a few things you have seen

Why deleting is instant, even for a huge file. It removes a name from a list. The bytes are still there; they are simply no longer claimed — which is why deleted files can sometimes be recovered.

Why a corrupted disk loses everything at once. Damage the list and the bytes remain, perfectly intact, with nothing left saying where any file starts or what it was called.

The pattern is the one from the bottom of the stack, again: the system describing the data is **made of** data. One mechanism, used on itself.

Why files live in a tree

Millions of files on one disk, each needing a unique name, is unworkable — two projects could never both have `notes.txt`.

The fix falls out of what a folder already is. If a folder is a list of names, one of those names can point at **another list** — a folder inside a folder, nesting as deep as you like, from one starting point at the top called the **root**.

A name then only has to be unique **inside its own list**. Every project can have its own `notes.txt`, and they never collide.

A path is the route through that tree

To identify a file you give the sequence of folders leading to it, separated by `/` :

`/Users/you/dsap/ledger.csv`

folder you project the file

Where the program is standing

Your program is always **in** one folder — its **working directory**. So it can take a shortcut and name a file relative to where it stands:

<code>ledger.csv</code>	in the folder I am in now
<code>data/ledger.csv</code>	into the data folder, then the file
<code>../ledger.csv</code>	up one level, then the file

That is a **relative path**, and it is where most beginner errors live: the same program works in one folder and fails in another, because a relative path is a different file depending on where you stand.

A chatbot cannot see any of this. It does not know your folders or where you are standing. Remember that in the workshop.

Two traps in the name

The extension is a convention, not magic. `.csv` does not make a file a spreadsheet — the bytes are unchanged. It is a hint to your computer about which program to open it with, and to you about what to expect. Rename it `.txt` and every byte is identical.

Windows hides extensions by default. Many of you have been looking at `ledger` for years when the file was really `ledger.csv` — or worse, `ledger.csv.txt`, which explains why it would not open. That default has cost more student hours than any concept in this course.

One word you now understand properly

A **library** is code somebody else already wrote, tested and published.

You met the word this morning as a reason to choose this language. Now you know what it buys: somebody wrote the bit-level arithmetic so that you could type `shares * price` and think about a portfolio instead of switches. That is the stacking principle, sold as a product.

Standing on it is not cheating. It is the entire job.

PART 4

Why data science

What can we learn from the records we keep?

From calculating a value to judging a result

We know how to calculate what a portfolio is worth.

Now suppose we have to decide whether an investment strategy is worth keeping. What evidence would justify that decision?

Our portfolio grew. Did the strategy work?

	Portfolio balance
Start of month	CHF 10,000
End of month	CHF 12,800

Would you keep this strategy? Vote now — wooclap.com/RSOMLOO.

A. Yes, it made money · B. No · C. We need more information

Hypothetical portfolio; all figures checked in the accompanying demo.

First, follow the money

We deposited **CHF 2,000** at the end of the month.

The balance grew by	CHF 2,800
Of which we paid in	CHF 2,000
Investment gain	CHF 800

The balance rose **28%**. The investment return was:

$$\frac{12,800 - 10,000 - 2,000}{10,000} = 8\%.$$

Same hypothetical portfolio. The deposit arrived after the investment return.

Then ask: compared with what?

Suppose a broad-market investment returned **10%** over the same month. Use it as our **benchmark**: the alternative we compare against.

Same starting money, same deposit	Closing balance
Our strategy	CHF 12,800
Benchmark	CHF 13,000

We made money, but finished **CHF 200 behind the alternative**.

Hypothetical benchmark; no fees or taxes in this example.

Now we can say something defensible

First impression	After looking at the records
"We made 28%."	The investment return was 8%.
"The strategy worked."	It trailed our benchmark this month.

We can now investigate whether the strategy deserves its cost.

One month does not establish skill. But we have a meaningful comparison instead of a misleading headline.

That work is data science

Use data to answer a question about the world.

In our example, that meant finding the relevant records, separating contributions from gains, and choosing a comparison that answered our question.

The computing made the calculation possible. The question determined which calculation was worth doing.

The payoff is a better decision

With holdings histories and transaction-and-fee records, we can investigate:

Question	What the answer informs
Where did the gains come from?	Which positions to examine
How much did trading cost us?	Whether to trade so often
Which holdings move together?	Whether we are really diversified

Each begins with a practical question, not a choice of algorithm.

Why there is so much opportunity now

A trading system already records transactions to keep the accounts. Those same records can also help evaluate a strategy.

Data collected for one job can answer another question.

Cheap storage keeps the history. Computing and reusable libraries let us examine it repeatedly, as new questions arise.

Some questions have examples, but no ready-made rule

The ledger tells us what happened. We might also want to predict what happens next.

We cannot write down a dependable rule for tomorrow's price merely because we can calculate today's portfolio value.

But we have past cases. For each, we can see what happened next and check how well a proposed rule would have done.

This is the opening for machine learning

Learn a rule from examples, rather than specify every part by hand.

The examples help choose the rule. We can repeat that process as new data arrives, instead of manually rewriting a long list of instructions.

The test of success is whether the learned rule works on new cases.

What questions can learning address?

What we want	A question about our ledger
Predict a number	What might the next price be?
Predict a category	Does this trade need review?
Find structure	Which assets behave similarly?

These sound like different jobs. A large part of machine learning comes from finding a common mathematical way to approach them.

Start with something adjustable

One possible **model**: a weighted average of the last two prices.

$$\hat{p}_{\text{next}} = a p_{\text{today}} + (1 - a) p_{\text{yesterday}}, \quad 0 \leq a \leq 1.$$

The p 's are prices; the hat marks a **prediction**.

The weight a is the setting we can change.

At $a = 1$, use today's price. At $a = 0$, use yesterday's.

The data will help us choose a .

Give the model a score

For each past date i , compare the prediction $\hat{p}_i(a)$ with the price p_i that actually occurred.

$$L(a) = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i(a) - p_i)^2.$$

Square each error, then average over the n dates we scored.

This is a **loss**. It depends on a because changing the weight changes the predictions.

A prediction problem becomes an optimization problem

Choose the settings that make the loss small:

$$\min_{\theta} L(\theta).$$

θ stands for the model's allowed settings. In our example, it is just the weight a between 0 and 1; other models have many settings.

A very large body of learning problems can be written in this form. That is what lets us use numerical **optimization** software.

Supervised learning: the examples supply answers

Information available at the time	Answer learned afterwards
Earlier prices	The next price
Details of a past trade	Whether it needed review

Learning from examples paired with known answers is **supervised learning**.

The loss measures disagreement with those answers. Adjusting the model to reduce it is called **training**.

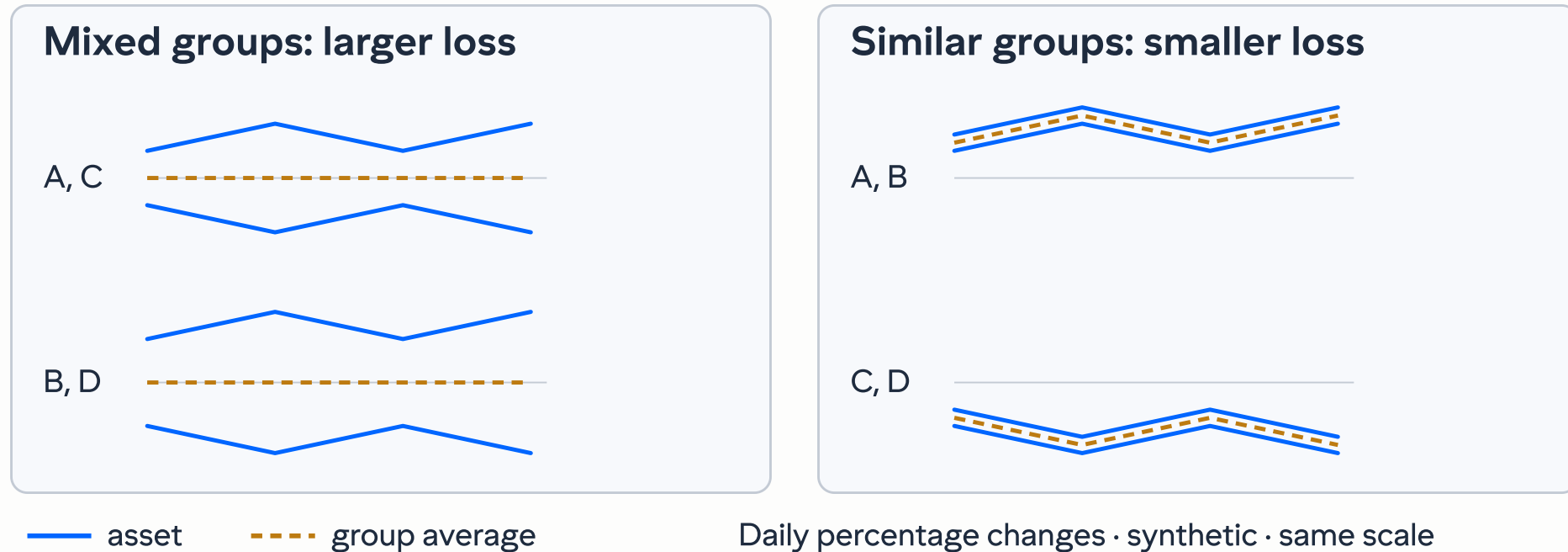
Unsupervised learning: find structure without an answer key

Take the same price histories. Group assets with similar patterns of daily percentage price changes.

Nobody has labelled the correct groups. We have to specify what makes a grouping useful.

This is **unsupervised learning**. It can reveal that apparently different holdings tend to behave alike.

We can give a grouping a loss too



Keep the number of groups fixed. The dashed line is each group's average pattern.

Square the gaps from each asset to its group average and add them up:
that is the loss we minimise.

The grouping objective, written down

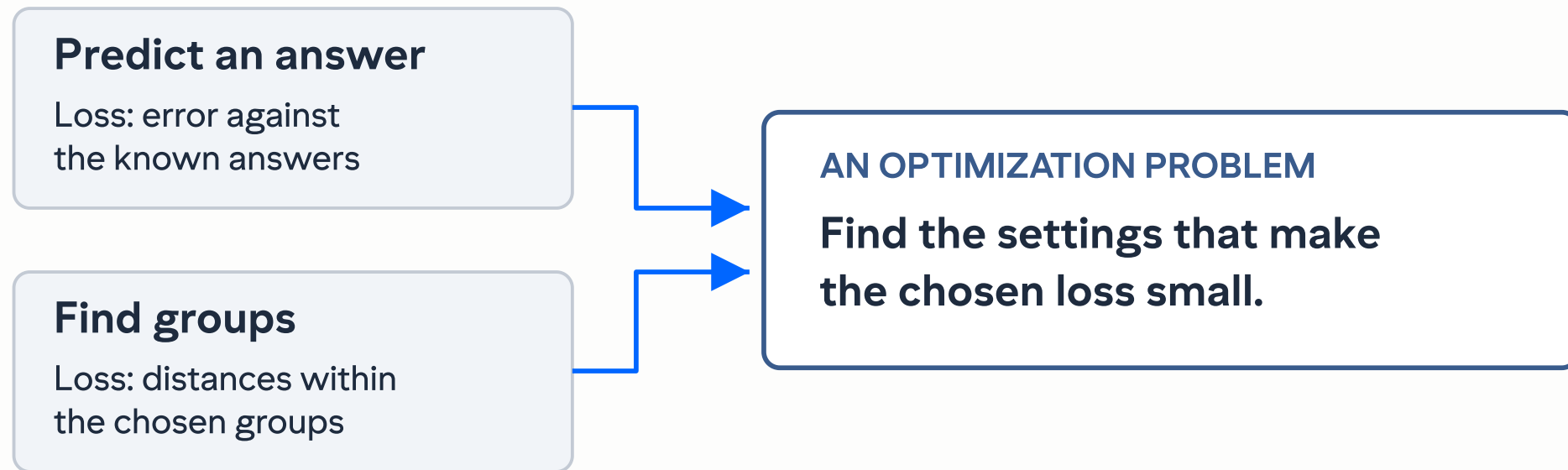
Split the assets into k nonempty groups. Keep k fixed.

$$L(G_1, \dots, G_k) = \sum_{g=1}^k \sum_{i \in G_g} \|x_i - \mu_g\|^2.$$

- G_g : the assets assigned to group g .
- x_i : asset i 's pattern; μ_g : its group's average pattern.
- $\|x_i - \mu_g\|^2$: square the gaps **on each day**, then add them.

Minimise this loss by choosing which assets belong together.

Different questions. The same mathematical task.



A major unifying idea in machine learning: **change the objective; reuse the machinery for minimising it.**

Why this reduction is such a good trade

Numerical optimization is one of the best-developed areas of numerical mathematics.

It offers studied methods, known conditions for their use, and well-developed software libraries.

We can spend our effort on the original problem instead of inventing its numerical machinery from scratch.

The stacking principle, one level higher

We decide	A suitable library provides
What the model may use	An implementation of the fitting method
What counts as a mistake	Machinery to reduce the chosen loss
How to check the result	Tools to measure its performance

We still own the question. We can reuse the machinery.

AI has pushed optimization research forward

Training a large AI model makes small improvements in efficiency valuable. Less work per training run buys more experiments with the same budget.

That demand has driven substantial work on optimization algorithms:
lower losses, less memory, less computation.

Machine learning benefits from optimization—and sends new problems and resources back into it.

Two examples of that traffic

Method	What motivated the work
Adam — ICLR 2015	Efficient training with many observations or model settings
Muon for LLM training — 2025	Adapting an optimizer to train large language models efficiently

You do not need to know how either works today.

Their existence matters: better optimization changes what we can afford to train.

Modern AI optimization relies on GPUs

Modern optimization for large neural networks relies more on **GPUs than CPUs** for its heavy numerical work.

Training repeats huge numbers of additions and multiplications on arrays. A GPU runs many of these calculations **in parallel**, across its many arithmetic units.

The CPU coordinates the work; the GPU carries the main arithmetic load.

Remember this link: large-scale AI training depends on GPU computing.

So is machine learning just statistics?

You will hear different answers:

- "They are the same subject."
- "Machine learning is statistics plus neural networks."
- "They overlap, but ask different questions."

There is no fully agreed boundary. These are competing descriptions, not definitions you need to choose between today.

Much of the machinery is already statistics

Machine-learning terminology	Statistical terminology
Features — the model's inputs	Predictors / covariates
Target / label — the answer to predict	Response / dependent variable
Training the model	Fitting / estimating parameters
Minimising squared-error loss	Least-squares fitting

$$\min_{0 \leq a \leq 1} \frac{1}{n} \sum_{i=1}^n (\hat{p}_i(a) - p_i)^2.$$

Our price model uses **least-squares regression**, under either subject's name.

The people crossed the supposed boundary too

Machine learning grew out of **statistics and computer science**.

There has never been a clean line separating the work.

Leo Breiman was a statistician. His **Random Forests** paper appeared in a journal called **Machine Learning**.

Scientists could work in both without changing the problem they were trying to solve. The subject labels did not fence the methods in.

The optimizer solves the problem we give it

We want **accurate predictions of future prices**.

We give the optimizer a different, measurable task:

Find the weight a with the **smallest error on recorded prices**.

Even if it finds the best possible weight, that only establishes a fit to those records. Will the same patterns hold on new dates? Did we leave important assets out of the data?

Optimization finds a fit. Statistical reasoning asks whether that fit is evidence we can trust.

To test prediction, use prices the model has not seen

A flexible model can fit **accidental patterns in old prices** that never repeat. This is **overfitting**: a good training score, poor new predictions.

Earlier dates: training	Later dates: testing
Use these prices to choose the model's settings.	Keep these prices out of fitting and model selection.
Measures how well the model fits its examples.	Measures how well it predicts unseen prices.

Fix the model first, then score its predictions on the later dates.

A good fit to the training data alone is not evidence of predictive skill.

A test on survivors cannot tell us about the failures

Suppose our price histories include **only companies still trading today**.

Companies that went bankrupt have disappeared from the dataset.

We split these histories into earlier training dates and later test dates.

Both parts still exclude the bankrupt companies.

The test may look good, but it never asks how the model would have behaved on those failures—assets we could have bought at the time.

Unseen data is not automatically representative data.

Check who is missing before trusting the score. The same problem appears in surveys.

Two surveys. Which would you trust?

Both measured first-dose COVID vaccination among US adults in early 2021.

Survey	Responses per week
Delphi-Facebook	~250,000
Axios-Ipsos	~1,000

Vote at wooclap.com/RSOMLOO: A. Delphi-Facebook · B. Axios-Ipsos · C. Size is not enough to tell

The larger survey missed by 17 points

In May 2021, Delphi–Facebook overestimated vaccination by **17 percentage points** relative to the revised CDC benchmark.

Its huge sample gave it **tiny reported margins of error** around that wrong estimate.

The smaller Axios–Ipsos survey tracked the benchmark much more closely.

Bradley et al., **Nature** (2021)

How much accuracy did those answers buy?

The researchers found that an estimate from **250,000 responses** could be no more accurate than one from a **simple random sample of 10**.

The comparison is about this estimate's accuracy.

A large number of responses did not compensate for who was missing.

Bradley et al., **Nature** (2021)

More responses reduce noise, not selection bias

Suppose vaccinated people are **more likely to answer** the survey. They will be **overrepresented among respondents**, so the raw survey estimate will tend to be too high.

Collect more answers in the same way: the estimate fluctuates less, but **the same group is still overrepresented**. This is **selection bias**.

The survey did use weighting to balance recorded characteristics. That cannot guarantee a correction if respondents and nonrespondents still differ in vaccination within those groups.

A larger sample can make us more certain of the wrong answer.

Averaging does not repair a faulty scale

A scale reads consistently heavy. We weigh the same object repeatedly and average the readings.

The random fluctuations settle down. The extra weight remains.

More data reduces random noise. A systematic bias needs a different fix.

The more data we have, the surer we fool ourselves.

Data abundance makes statistical care more important

A bigger dataset can make an answer look more certain without making it more accurate.

Ask who is missing. Check against an independent source. State what the uncertainty calculation does—and does not—account for.

The opportunity is to learn more from data. The responsibility is to establish what the data actually supports.

What we are learning to do

	The job
Data science	Turn records into evidence for a useful question
Machine learning	Learn predictions or structure from examples
Optimization	Fit those choices by minimising an objective
Statistics	Judge what the observations and results justify

These are connected activities, not separate boxes.
We want to produce an answer and be able to defend it.

Two books worth knowing

Kevin Murphy — Probabilistic Machine Learning: An Introduction.

An excellent introduction to statistics inside a machine-learning book.

A good place to see why the subjects belong together.

Larry Wasserman — All of Statistics: A Concise Course in Statistical Inference. A great, compact introduction to the statistical foundations.

Brisk rather than gentle.

Neither is required. Murphy provides a free draft on his [book website](#).

What you can now do

- Say what a variable is, and why `=` is not equality.
- Trace a short program by hand and predict its output.
- Explain why a program that runs is not a program that is right, and name the three ways code fails.
- Say what a transistor is, and why one switch throwing another is the whole reason a computer can exist.
- Explain how gates are built from switches and everything else from gates — and why `0.1 + 0.2` was never Python's fault.
- Say what a bit, a byte, a file and a path are.
- Connect data science, machine learning, optimization and statistics.

Next

Next session: types, text, and arithmetic that behaves — plus what to do when line 3 explodes.

Workshop this afternoon: get your machine working, run your first program, and break it on purpose.

You do not learn to read code by reading code.

You learn to read code by writing it.

— the whole course, in one sentence